Airflow ETL Pipeline with S3 Sensors, PostgreSQL, and Slack Alerts

1. Introduction

This project demonstrates the implementation of a production-style ETL (Extract, Transform, Load) pipeline using Apache Airflow running in Docker. The pipeline monitors AWS S3 for incoming files, performs transformations and aggregations, stages processed data back to S3, loads data into PostgreSQL, and sends Slack notifications after successful execution.

---

2. Project Objective

The main objective of this project is to:

- Detect multiple files arriving in AWS S3 between a specific time window
- Monitor files in parallel using Airflow Sensors
- Perform transformation and aggregation on incoming data
- Store processed output into staging blob storage
- Load transformed data into PostgreSQL database
- Send pipeline execution alerts using Slack
- Run the entire workflow using Dockerized Airflow

---

3. Technologies Used

Technology	Purpose
Apache Airflow	Workflow orchestration
Docker	Containerized environment
AWS S3	Blob/Object storage
PostgreSQL	Target database
Pandas	Data transformation
Slack Webhook	Notifications and alerts
Python	DAG and ETL logic

4. Architecture Overview

AWS S3 (Raw Files) —> Parallel S3 Sensors → Transformation & Aggregation → Staging S3 Location → PostgreSQL Database → Slack Notification

---

5. Project Directory Structure

```
airflow-docker/
│
├── dags/
│   └── parallel_file_ingestion_pipeline.py
│
├── logs/
├── plugins/
├── config/
├── requirements.txt
├── docker-compose.yaml
└── .env
```

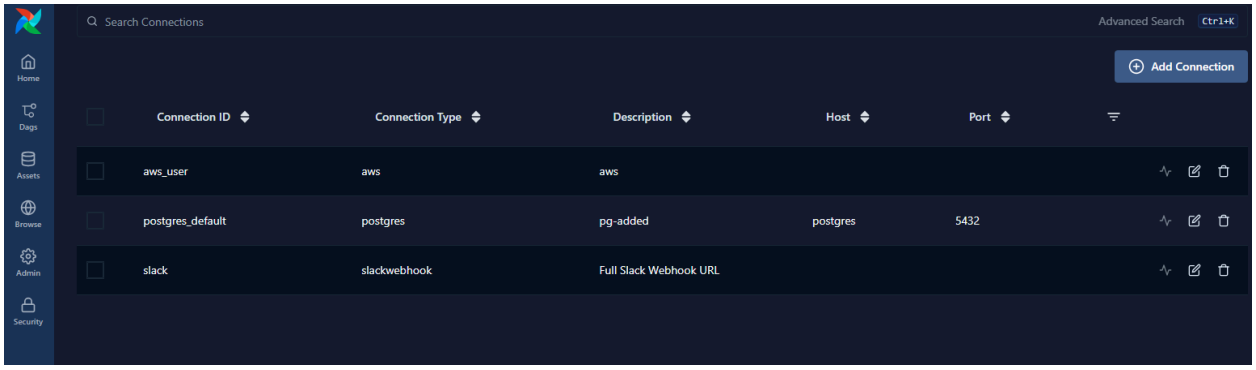
---

6. Docker PostgreSQL Configuration

```
postgres:
 image: postgres:16
 ports:
   - "5432:5432"
 environment:
   POSTGRES_USER: airflow
   POSTGRES_PASSWORD: airflow
   POSTGRES_DB: airflow
```

---

7. Airflow Connections



AWS Connection

Field	Value
Connection Id	aws_default
Connection Type	Amazon Web Services

PostgreSQL Connection

Field	Value
Connection Id	postgres_default
Connection Type	Postgres
Host	postgres
Database	airflow
Username	airflow
Password	airflow
Port	5432

Slack Connection

Field	Value
Connection Id	slack_webhook
Connection Type	Slack Incoming Webhook

8. Airflow Variables

Key	Value	Description	Is Encrypted
bucket_name	raw-data-bucket		false
file1_key	sales/sales.csv		false
file2_key	customer/customer.csv		false
staging_key	staging/output.csv		false
table_name	sales_summary		false

Variable	Value
bucket_name	airflow-demo-bucket-ps
file1_key	sales/sales.csv
file2_key	customer/customers.csv
staging_key	staging/output.csv
table_name	sales_summary

9. Workflow Execution Steps

Step 1: DAG Trigger

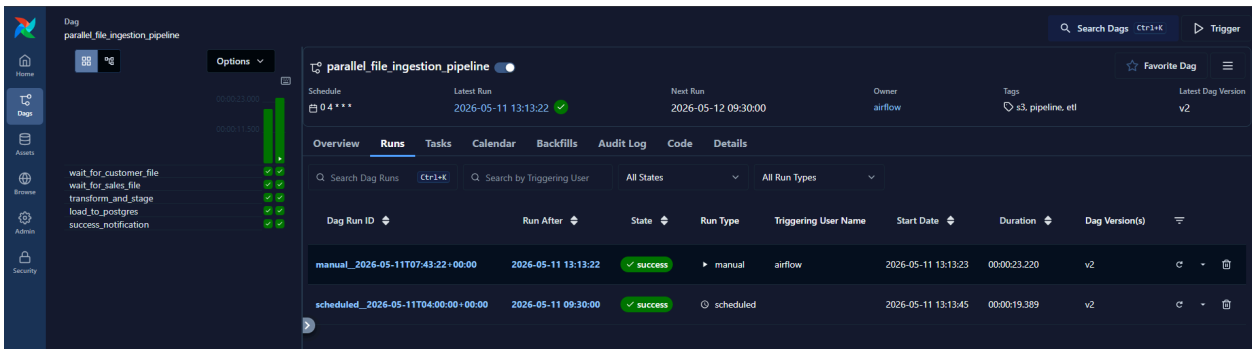
The DAG starts execution at the scheduled time.

Step 2: Parallel File Detection

Two S3KeySensors run in parallel:

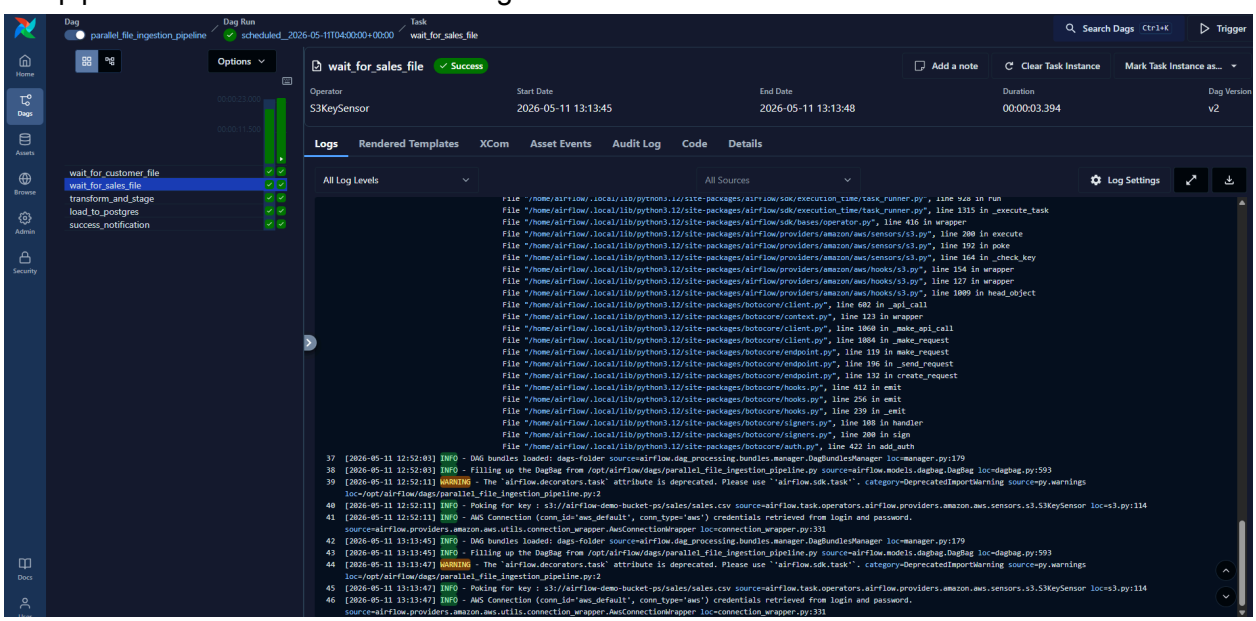
- Detect `sales/sales.csv`
- Detect `customer/customers.csv`

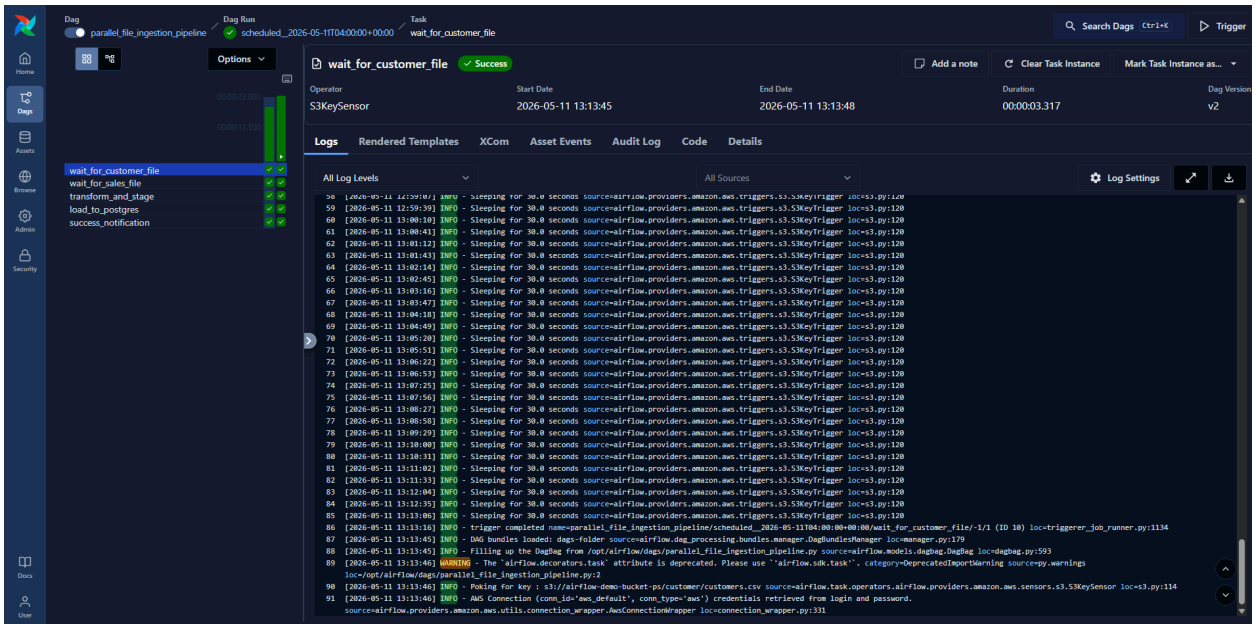
The DAG waits until both files are available in S3.



Step 3: Read Files from S3

The pipeline reads both CSV files using Airflow S3Hook.





Step 4: Data Transformation

The following operations are performed:

- Merge datasets using `customer_id`
- Aggregate sales amount by region
- Create transformed output dataframe

Example Logic:

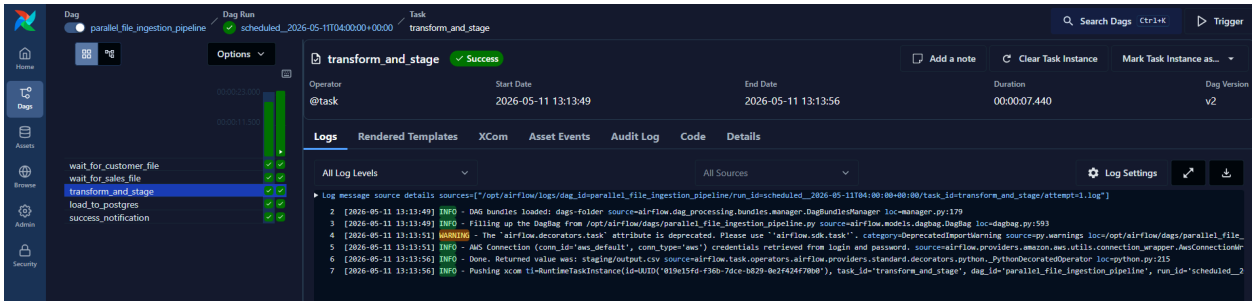
```
merged_df = sales_df.merge(
  customer_df,
  on="customer_id",
  how="inner"
)

aggregated_df = (
  merged_df.groupby("region")
  .agg({"amount": "sum"})
  .reset_index()
)
```

10. Stage Processed File

The transformed data is uploaded back to S3 staging location:

staging/output.csv



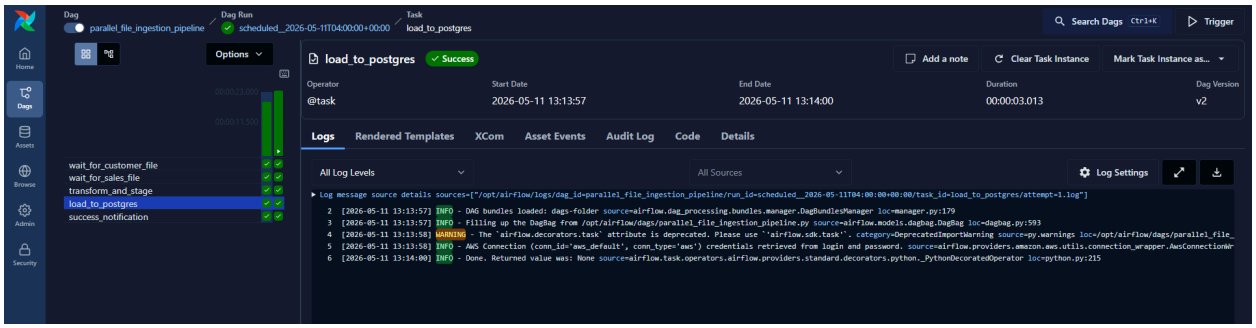
---

11. Load Data into PostgreSQL

Processed data is loaded into PostgreSQL table:

sales_summary

using Pandas `to_sql()` method.

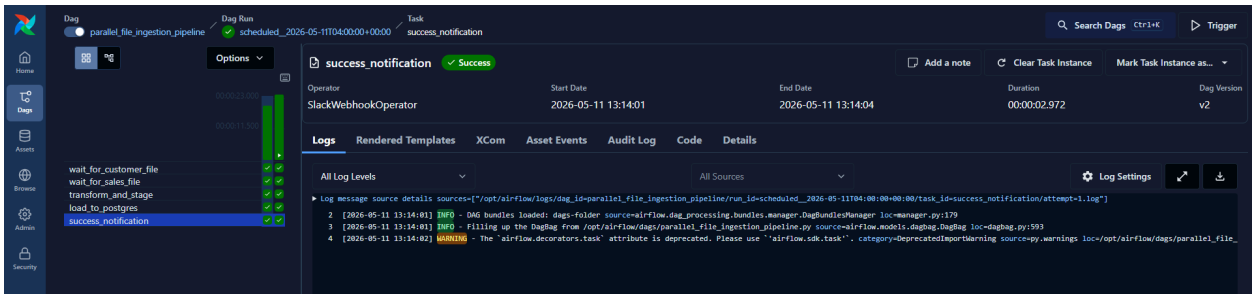


---

12. Slack Notification

After successful pipeline completion, Slack notification is sent:

Pipeline completed successfully
DAG: parallel_file_ingestion_pipeline



---

13. Sample Input Files

sales.csv

customer_id,amount,region
1,100,West
2,200,East
3,300,West

---

customers.csv

customer_id,name
1,John
2,Alice
3,Bob

---

14. Expected Output

region,amount
East,200
West,400

---

15. Key Airflow Features Used

- S3KeySensor
- Parallel task execution
- Deferrable sensors

- TaskFlow API
- Airflow Variables
- Airflow Connections
- Retry mechanism
- Slack integration
- Dockerized deployment

16. Issues Faced and Resolutions

Issue	Resolution
AWS credentials not found	Created `aws_default` connection
PostgreSQL authentication failed	Updated correct DB credentials
Sensor waiting indefinitely	Uploaded correct file names
Slack webhook missing	Configured Slack Incoming Webhook
Deferred task issue	Added triggerer or changed sensor mode

17. Advantages of This Pipeline

- Automated file monitoring
- Parallel execution improves efficiency
- Scalable architecture
- Real-time alerts
- Cloud storage integration
- Enterprise-style orchestration
- Easy deployment using Docker

18. Conclusion

This project successfully demonstrates an end-to-end ETL orchestration workflow using Apache Airflow with AWS S3, PostgreSQL, Slack integration, and Docker. The implementation includes parallel sensors, transformations, staging architecture, database loading, and automated notifications, making it suitable for real-world enterprise data engineering use cases.